LocalFix — End-of-year report

All-in-one patch blocks

Checklist (apply in your main report)

- **Block A** — Chapter 2, \subsection{Sprints breakdown}: replace the two \item lines for Sprint 3 and Sprint 4.
- **Block B** — Chapter 2 product backlog table (optional): insert before \end{tabular} after row ID 9.
- **Block C** — Chapter 4 \chapter{Release 2}: replace only the *body* of \section{Introduction} (keep the section command if you already have it).
- **Block D** — Chapter 4: remove the old \section{Sprint 3...} through the line before \section{Sprint 4...}; paste Block D there.
- **Figures** — Add to your project: sprint3-usecase.png, sprint3-class.png, sprint3-sequence-accep sprint3-sequence-complete.png, sprint3-sequence-admin.png, sprint3-screens-tasks.png, sprint3-screens-notify.png, sprint3-screens-map.png, sprint3-screens-admin.png.
- **Optional:** rename \subsection{Sprint 1 Objective } to \subsection{Objective of Sprint 1} for consistency with other sprints.

1 Block A — Chapter 2 sprint breakdown (paste inside itemize)

```
\item Sprint 3 -- Interaction and status: implement ``Offer help'' / accept task and ``Mark

\item Sprint 4 -- Reviews, persistence, and polish: implement ``Rate helper'' after completi
```

2 Block B — Optional product backlog rows (before \end{tabular})

```
\hline
10 & Map \& discovery & As a member, I want to see helpers on a map with filters so I can ch
\hline
11 & Trust \& safety reports & As a member, I want to file structured reports with evidence
\hline
12 & Admin console & As an administrator, I want a dashboard for tasks, accounts, and report
```

3 Block C — Release 2 introduction (replace section body only)

```
\section{Introduction}
This chapter documents \textbf{Release~2} of LocalFix, structured like \textbf{Release~1}: e
```

4 Block D — Sprint 3 full section (same structure as Sprints 1–2)

Below is the live LaTeX you paste before your existing `\section{Sprint 4}`. In *this* document, figures show as black rectangles because of the `demo` option.

5 Sprint 3: Interaction, task status, and platform visibility

5.1 Objective of Sprint 3

The objective of the third sprint is to connect the task objects created in Sprint 1 with the communication channels delivered in Sprint 2. Helpers must be able to commit to an open task, both parties must share the same understanding of progress through explicit states, requesters must close the loop by marking work complete and (in a later increment) leaving feedback, and the platform must signal important events through in-application notifications. In parallel, trust-and-safety data entered through the reporting flow gains a moderation lifecycle (open, assigned, resolved), and the administrator role gains read-only visibility over the live task register and account list so operators can supervise the neighborhood marketplace during demonstrations.

5.2 Backlog of Sprint 3

Id	Feature	User Story	Priority	Complexity
1	Offer help / accept task	As a Helper, I want to offer help on an open task so that the requester knows I am available and the task moves to an in-flight state.	High	Medium
2	Mark task complete	As a Help Requester, I want to mark a task as completed once the helper has finished so that the lifecycle closes cleanly.	High	Medium
3	Task status visibility	As a user, I want to see whether a task is open, in progress, or completed so that everyone shares the same understanding.	High	Low
4	In-app notifications	As a user, I want notifications when someone accepts my task, sends a message, or when the administrator broadcasts an update so that I do not miss important events.	High	Medium
5	Map and discovery (atlas)	As a member, I want to browse helpers on a map with filters so that I can choose someone nearby before starting a conversation.	Medium	Medium
6	Report moderation	As an Administrator, I want to assign, verify, and resolve reports with notes so that trust-and-safety issues are tracked transparently.	High	High
7	Admin dashboard	As an Administrator, I want aggregate metrics plus tables of tasks and accounts so that I can supervise the session during demos.	Medium	Medium

Table 1: Sprint 3 product backlog

5.3 Analysis of Sprint 3

Figure 4.1 presents the use-case diagram for Sprint 3, extending the actors introduced in earlier sprints: Help Requester, Helper, and Administrator now interact with task-state transitions, the notification center, the map-based discovery view, and the moderation workspace for submitted reports.



Figure 1: Sprint 3 use case diagram

Description of scenarios:

Offer help on a task

1. The helper opens the task board and locates an open task that matches their skills or availability.
2. The helper chooses *Offer help* (or equivalent action) on that task.
3. The system checks that the helper is not the requester, that the task is still open, and that the actor is not blocked by role rules (for example, the operator account in Sprint 4 cannot accept tasks).
4. The system transitions the task to an in-progress state, records the helper name, and notifies the requester.

Mark task complete

1. The requester opens an in-progress task where they are the owner.
2. The requester selects *Mark complete* (or equivalent).
3. The system validates that the lifecycle allows completion, updates the status to completed, and notifies the helper.
4. The interface may prompt for a rating in a later sprint once the demonstration flow reaches Release 2 / Sprint 4 scope.

Consult notifications

1. The user opens the notifications view from the main navigation.
2. The system lists events targeted to the signed-in account (task accepted, message received, broadcast, new report for administrators, etc.).
3. The user marks items as read; unread counts appear in the shell header until cleared.

Moderate a report (administrator)

1. The administrator opens the console and reviews the report queue.

2. The administrator assigns a moderator name, toggles verification, or resolves the case with a resolution note.

3. The system stores the updated status and preserves the narrative for audit purposes inside the browser session.

5.4 Design of Sprint 3

a – Class diagram of Sprint 3 :
Sprint 3 refines the associations started in Sprint 1 and Sprint 2. The **Task** aggregate now exposes explicit state transitions and optional attachment metadata; the **User** aggregate differentiates standard members from the **Administrator** profile through a role flag; **Report** entities capture moderation fields (status, assignee, resolution note, verification flag); **Notification** entities encapsulate user-targeted messages with read state and timestamps; and the map experience reuses **Helper** profile data with geospatial attributes for presentation. Sequence interactions focus on the application layer mediating between the React view, in-memory repositories, and the notification subsystem.



Figure 2: Sprint 3 class diagram

b – Sprint 3 sequence diagrams :
Figure 4.3 illustrates the "Offer help" interaction among the helper client, front-end controller, task aggregate, and notification service. Figure 4.4 covers the "Mark complete" path for the requester. Figure 4.5 summarizes the administrator moderation path for a submitted report.



Figure 3: Sprint 3 sequence diagram — Offer help / accept task



Figure 4: Sprint 3 sequence diagram — Mark task complete



Figure 5: Sprint 3 sequence diagram — Report moderation

5.5 Completion of Sprint 3

To better understand how the third increment behaves in the demonstrator, this subsection highlights the main interfaces that realize the Sprint 3 backlog: the task board with apply and completion controls, the notification center, the atlas view with helper markers, and the administrator console with the task register, account table, and report queue. Figures 4.6–4.9 should be replaced with screenshots exported from the current React client.

Task board and lifecycle :



Figure 6: Task board showing open and in-progress tasks (Sprint 3)

Notifications :

Map / discovery :

Administrator console :



Figure 7: In-app notifications (Sprint 3)



Figure 8: Atlas view with helper markers (Sprint 3)



Figure 9: Admin console with metrics, task register, and reports (Sprint 3)